

Privacy-Preserving Fuzzy Name Matching

项目分工与接口规范

关键参数: $EL=200/50$ · $k \approx \sqrt{N}$ · $\tau=0.9$ · $\text{poly_modulus}=8192$

01

MinHash 与姓名编码

数据预处理 · Shingle 生成 · MinHash 签名 · L2 归一化

数据流

- 上游 原始姓名列表, 来自数据集文件 (Party A 的查询名单, Party B 的数据库名单)
- 下游 两张归一化签名矩阵: **EL=200** (交给成员 4 做聚类) 和 **EL=50** (交给成员 3 做列匹配)

核心任务

- 小写化、去空格, 统一名字格式
- 按 trigram 滑窗切分 (含首尾空格) 生成 shingle 集合
- 用 SHA-256 对每个 shingle 取哈希, $\text{mod } 2^{20}$
- 应用 200 个线性置换取最小值, 生成 MinHash 签名
- EL=50 签名直接取前 50 维, 保证置换一致性
- 对两张矩阵分别逐行 L2 归一化

对外暴露接口

```
clean_name(name) → str    build_shingles(name, size=3) → set  
generate_signature(name, num_perms) → ndarray  
batch_encode(names, num_perms) → ndarray (N, d)    l2_normalize(matrix) → ndarray
```

↓

数据流

- 上游
- 来自成员 1 的归一化签名向量 (numpy 数组) , 以及来自成员 3 需要加密或解密的任意数值向量
- 下游
- 加密后的密文对象和序列化字节, 供成员 3 做同态运算; 解密结果还给 Party A 逻辑层

核心任务

- 初始化 TenSEAL CKKS 上下文, 参数固定为 `poly_modulus=8192`、`scale=240`、`coeff_mod=[60,40,40,60]`
- 生成公钥与私钥, 以及同态乘法所需的重线性化密钥
- 将 `numpy` 向量加密为 `CKKSVector`, 并支持批量 `slot packing`
- 将密文序列化为字节流, 用于模拟网络传输; 反向反序列化
- 持有私钥并提供解密接口, 私钥不离开 Party A

对外暴露接口

```
build_context() → ts.Context    generate_keys(ctx) → (pk, sk)

encrypt(vec, ctx) → CKKSVector  decrypt(ct, sk) → ndarray    serialize_ct(ct) → bytes

deserialize_ct(data, ctx) → CKKSVector
```



数据流

- 上游
- 成员 2 提供的密文对象（加密查询向量），以及成员 4 提供的明文质心或列矩阵数据
- 下游
- 每列的加密分数 $E(\text{score}_j)$ ，返回给 Party A 解密后判断 $\text{catch}=0/1$

核心任务

- CT-PT 点积：用于质心匹配（Step 3）和列选择（Step 6），密文乘明文，逐元素相乘后累加
- CT-CT 点积：用于余弦相似度计算（Step 7），两个密文直接相乘累加，需重线性化
- 从加密分数中减去阈值 $\tau=0.9$ （明文常数加法，无噪声代价）
- 乘以每列独立随机正整数 r_j ，混淆真实相似度数值，保护 Party B 的隐私

对外暴露接口

```
dot_ct_pt(ct, pt_vec) → CKKSVector    dot_ct_ct(ct1, ct2) → CKKSVector
subtract_threshold(ct, tau) → CKKSVector    apply_mask(ct) → CKKSVector
```



数据流

上游 成员 1 提供的两张归一化矩阵 (EL=200 用于聚类, EL=50 用于列矩阵), 以及成员 3 的点积算子

下游 离线产物 (质心 C_k 、列矩阵 C 、scaler 参数) 存入 artifacts/; 在线阶段输出逐列加密分数交给成员 3

核心任务

- 对 EL=200 矩阵拟合 StandardScaler, 均值和标准差作为公开参数共享给 Party A
- 在标准化后的矩阵上运行余弦 K-Means ($k \approx \sqrt{N}$, 迭代 20 次)
- 按聚类编号将 EL=50 签名重排成列矩阵 C , 不足 max_size 的 cluster 用零向量填充
- 在线阶段编排 Party B 的两轮响应: 质心比较 → 逐列匹配, 支持流式发送

对外暴露接口

```
fit_scaler(matrix) → scaler    transform(matrix, scaler) → ndarray  
run_kmeans(matrix, k) → (centroids, assignments)  
build_column_matrix(sigs_50, assignments) → ndarray (k, max_size, 50)  
compare_to_centroids(ct_query, centroids) → list[CKKSVector]  
column_wise_match(ct_q50, ct_S, col_matrix) → generator[CKKSVector]
```



数据流

- 上游
- 原始数据集文件（NCVR、图书馆目录、US Census），以及协议各阶段输出的 catch 值、密文对象和计时数据
- 下游
- 面向论文和团队的结果报告：精度指标、通信开销表、各配置对比数据

核心任务

- 加载三类数据集，构建 A 与 B 的名单以及 ground truth 匹配标签
- 计算 Accuracy、Precision、Recall、F1
- 用 TenSEAL 序列化测量每个密文的字节大小，统计各步骤通信量
- 测量端到端运行时间与内存峰值
- 消融实验：分别变化 EL、k、 τ 、数据集规模，记录指标变化

对外暴露接口

```
load_dataset(name, path) → (names_A, names_B, labels)

compute_metrics(preds, labels) → dict    measure_ct_size(ct) → int (bytes)

benchmark(config) → dict
```